AI-ASSISTED PASSENGER BELONGINGS RECOVERY SYSTEM

MINI PROJECT REPORT

Submitted by:

K M ARIF (23011102036)
NITISH M (23011102057)

Project Mentored By: Dr. K. Surya

Course Instructed By: Dr. Shikha Marwaha

SEMESTER 6

BACHELOR OF TECHNOLOGY
IN
INTERNET OF THINGS

SHIV NADAR
UNIVERSITY
CHENNAI

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SHIV NADAR UNIVERSITY CHENNAI

APRIL 2026

K. Surya
(Dr. SURYA.K)
Supervisor.

ABSTRACT

The AI-Assisted Passenger Belongings Recovery System is a web-based platform designed to automate and optimize the process of recovering lost items on public transport networks such as state operated bus services. Traditional lost and found workflows rely on manual paper registers and telephone-based coordination, which are slow, error-prone, and geographically limited. This system digitizes and intelligently connects lost-item reports submitted by passengers with found-item logs recorded by depot staff. Using Natural Language Processing, the platform computes semantic similarity between textual descriptions to rank potential matches. A directional luggage path filter further eliminates geographically impossible match candidates. The result is a smart, centralized platform that dramatically improves item recovery rates for organizations such as TNSTC, connecting passengers and depot managers through an intuitive web interface.

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

1.1 Overview

The loss of personal belongings during public transport journeys is a widespread problem affecting millions of commuters and intercity bus passengers every day. In state-run transport networks such as the Tamil Nadu State Transport Corporation (TNSTC), passengers traveling across long routes for example, from Chennai to Coimbatore or Madurai to Tirunelveli occasionally leave items behind on buses. The current mechanism for recovering such items relies almost entirely on manual paper-based registers maintained at individual bus depots. Passengers must physically visit or call the relevant depot, describe their lost item verbally, and hope that a staff member can locate a match in a handwritten ledger. This approach is inefficient, geographically limiting, and results in a very low item recovery rate.

The AI-Assisted Passenger Belongings Recovery System addresses this critical gap by providing a centralized, intelligent web platform. Passengers can submit a digital lost-item report from any device, describing the item and their travel details. Depot staff, on the other end, log found items through a secure dashboard. An AI engine then cross-references these reports using Natural Language Processing (NLP) techniques to compute similarity scores and suggest the most probable matches. The system thereby transforms a slow, decentralized, paper-based process into a fast, scalable, and intelligent digital service.

1.2 Problem Statement

In the domain of public transport management, conventional lost-and-found operations struggle with two primary challenges: capturing the semantic meaning of item descriptions provided by passengers, and accounting for the directional geography of bus routes that physically constrains where a forgotten item can possibly be found. Traditional manual systems rely on exact keyword matching or human memory, which fails when passengers use different words to describe the same object for example, 'navy rucksack' versus 'blue bag'. Furthermore, without an understanding of bus route order, a system might incorrectly flag an item found at a depot before a passenger's boarding point as a potential match, wasting staff time and misleading passengers.

The proposed system resolves these fundamental limitations through a two-factor AI scoring model that combines text semantics and route validity into a single unified match score, enabling both accurate and geographically aware item recovery.

1.3 Objectives

The primary objectives of this project are:

1. To develop a web-based platform that digitizes and centralizes the lost-and-found process for public bus transport networks.

2. To implement a directional luggage path filter that eliminates geographically impossible item matches based on bus route stop indices.

3. To apply NLP-based semantic text matching using Sentence-BERT (sentence-transformers) to compare passenger descriptions with depot staff notes beyond simple keyword matching.

4. To design a unified scoring engine that combines text similarity (weighted) and route validity into a single match confidence score.

5. To provide passengers with a unique Tracking ID and a real-time status update indicating which depot holds their identified item.

6. To reduce the time and manual effort required for depot staff to identify item owners through an AI-ranked match dashboard.

1.4 Motivation

The motivation behind this project stems from the real-world inefficiency and human cost of current lost-and-found practices in public transport. Every year, a significant number of passengers lose valuables including identity documents, medicines, laptops, and personal belongings on long-distance buses. For many passengers, particularly those in rural areas or without easy access to communication, recovering a lost item is practically impossible under the current system.

The rapid advancement of transformer-based NLP models offers an unprecedented opportunity to bridge this communication gap intelligently. By applying these technologies in a lightweight Flask-based web application backed by MongoDB, a practical and deployable solution can be built without requiring expensive hardware or cloud infrastructure. This project is motivated by the desire to demonstrate that state-of-the-art AI can be applied to solve everyday civic problems, improving the quality of public services and restoring lost property to its rightful owners.

CHAPTER 2

BACKGROUND

2.1 Existing Lost and Found Approaches

Lost-and-found management in public transport has historically relied on entirely manual processes. Depot staff maintain physical registers in which found items are logged by hand with brief descriptions, dates, and route numbers. Passengers seeking lost items must telephone or physically visit individual depots, describe their item, and rely on a staff member to manually search through ledger entries for a potential match. This system functions reasonably well in small, low-traffic environments, but it fails at scale. State transport networks such as TNSTC operate hundreds of routes and dozens of depots simultaneously, making centralized manual recovery infeasible.

Some digitized systems have emerged in airports and railway networks, where items are logged into spreadsheet databases and queries are handled by a central help desk. However, these systems still rely on rigid keyword searches and do not account for semantic variation in item descriptions. A passenger who describes their lost item as a 'dark blue shoulder bag' will receive no match if the depot staff logged the same item as a 'navy satchel.' Furthermore, none of these approaches incorporate route-based geographic filtering, leading to irrelevant or impossible matches.

2.2 Overview of AI Matching Techniques

The application of AI to item matching in this system draws primarily from Natural Language Processing for semantic text similarity.

Sentence-BERT (Sentence Transformers)

Sentence-BERT is a modification of the BERT architecture that produces semantically meaningful sentence embeddings. Unlike traditional keyword-based search, Sentence-BERT encodes the full contextual meaning of a sentence into a dense numerical vector (embedding). Two sentences with similar meanings will produce embeddings that are geometrically close in high-dimensional space, even if they share no common words. This property is especially valuable for matching item descriptions, where passengers and staff independently describe the same object using different vocabulary. Cosine similarity between embedding vectors provides a reliable and computationally efficient measure of semantic closeness.

Cosine Similarity

Cosine similarity is the primary mathematical measure used throughout this system to quantify similarity between vector embeddings. It calculates the cosine of the angle between two vectors in multi-dimensional space, returning a value between 0 (completely dissimilar) and 1 (identical). It is particularly well-suited for high-dimensional embedding spaces because it is invariant to vector magnitude, focusing purely on directional alignment. The text similarity engine (Sentence-BERT) uses cosine similarity as its core scoring function.

Luggage Path Logic

A novel contribution of this system is the directional luggage path filter. Since a forgotten item can only move forward on a bus route from the point it was left, the system encodes each route as an ordered array of stop names. Given a passenger's alighting stop index within that array, the system automatically excludes any depot whose index is less than or equal to the passenger's stop as those depots are behind the item's last known location and are therefore physically impossible match candidates.

CHAPTER 3

METHODOLOGY

This chapter presents a comprehensive overview of the methodologies used to design, implement, and evaluate the AI-Assisted Passenger Belongings Recovery System. Each methodological step is described in detail below, from data collection through to the AI scoring pipeline and system architecture.

3.1 Data Collection

Data collection in this system occurs through two structured input channels: the passenger lost-item form and the depot staff found-item form.

3.1.1 Passenger Lost-Item Reports

Passengers access a public-facing web portal and submit a lost-item report. The form captures the passenger's name, phone number, the selected bus route, travel date, boarding stop, alighting stop, and a free-text description of the lost item. The interface also features an interactive Leaflet.js map on the right panel that displays all available TNSTC routes visually, allowing passengers to select stops directly from the map. Figure 3.1 shows the passenger lost luggage report form.

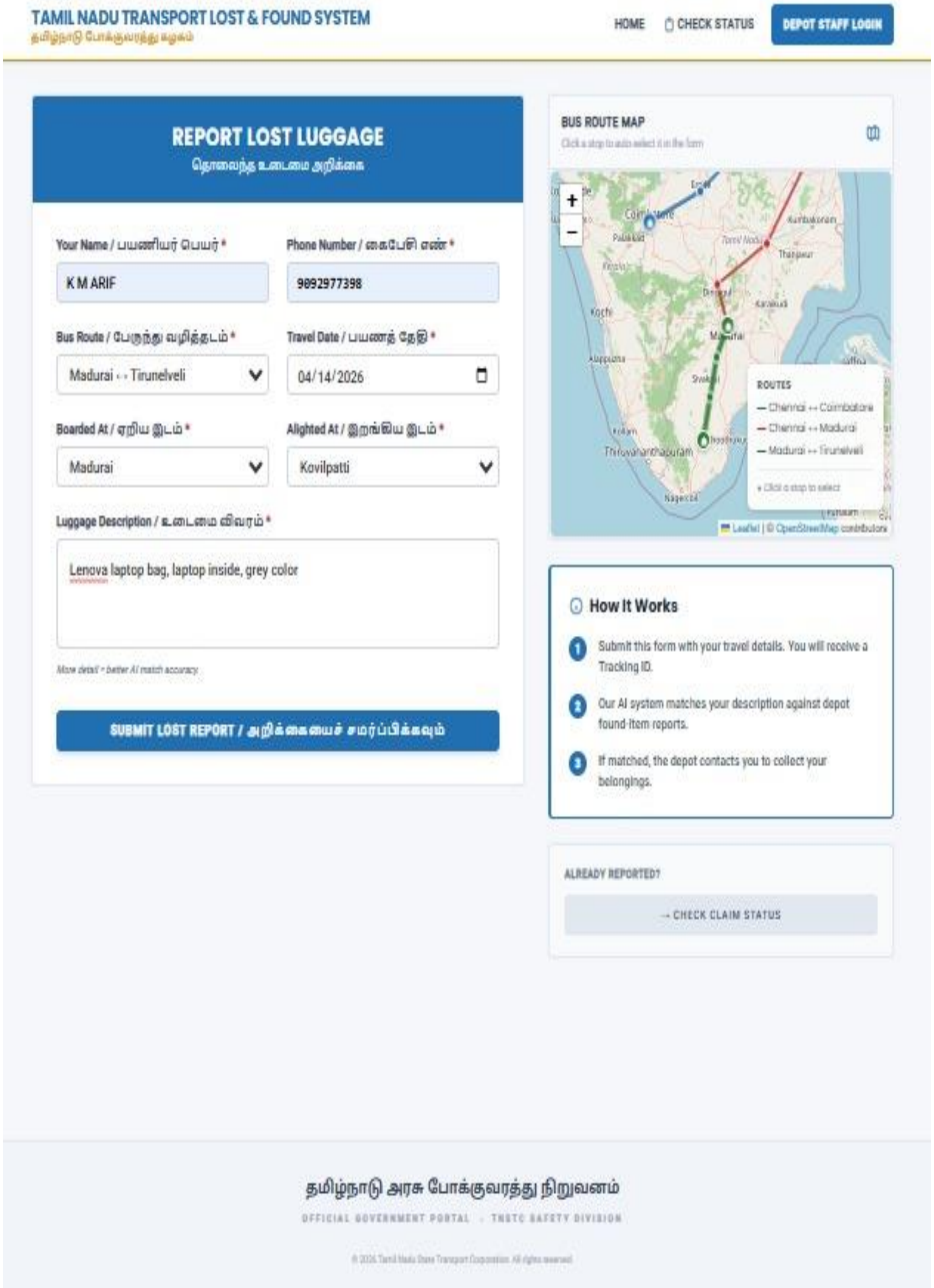

Fig 3.1: Passenger Lost Luggage Report Form

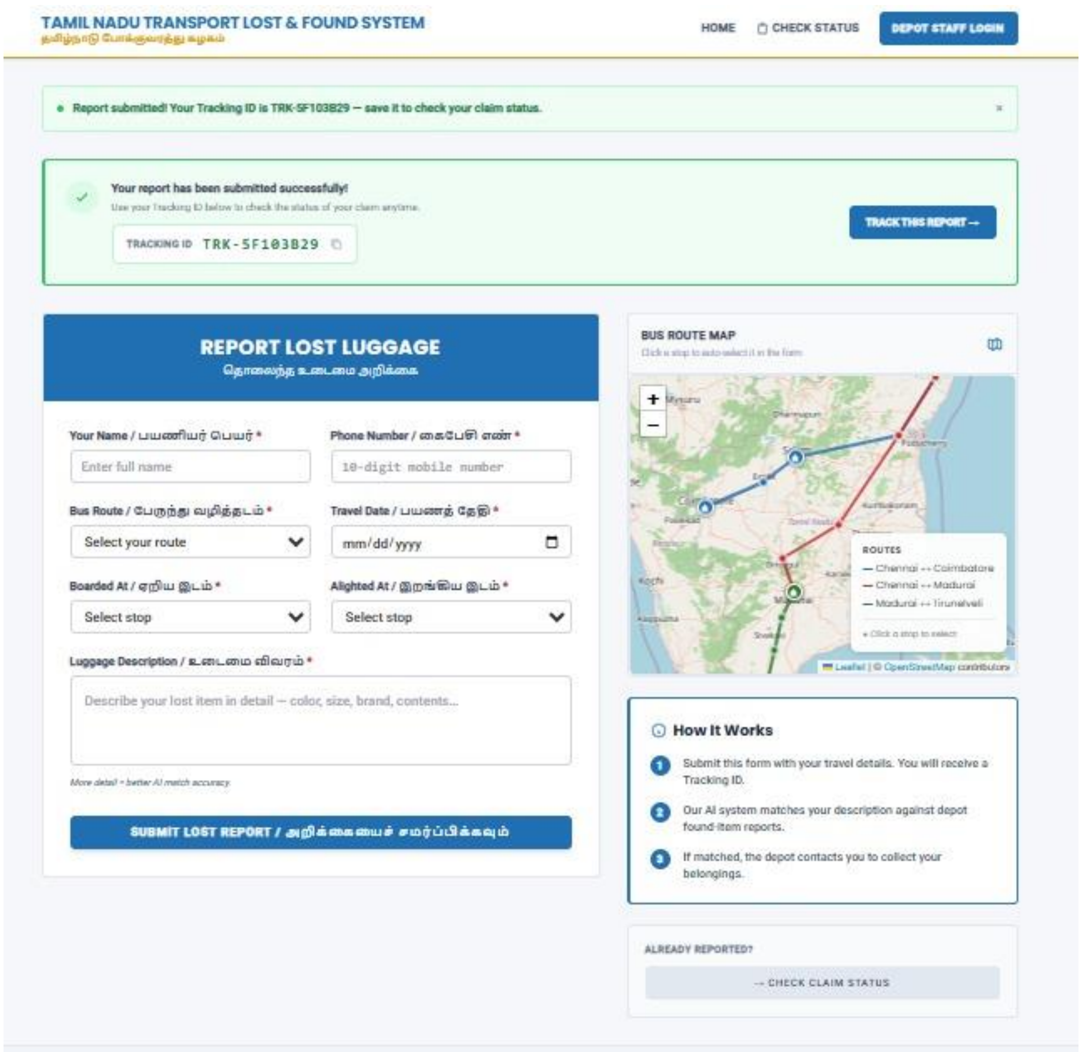

Fig 3.2: Tracking ID generation

Upon successful submission, the system generates a unique Tracking ID (e.g., TRK-5F103B29) which the passenger can use to monitor the status of their report at any time. A confirmation banner is displayed immediately, as shown in Figure 3.2. The given Figure 3.3 shows the Pending Status of the report being given in the input report form.

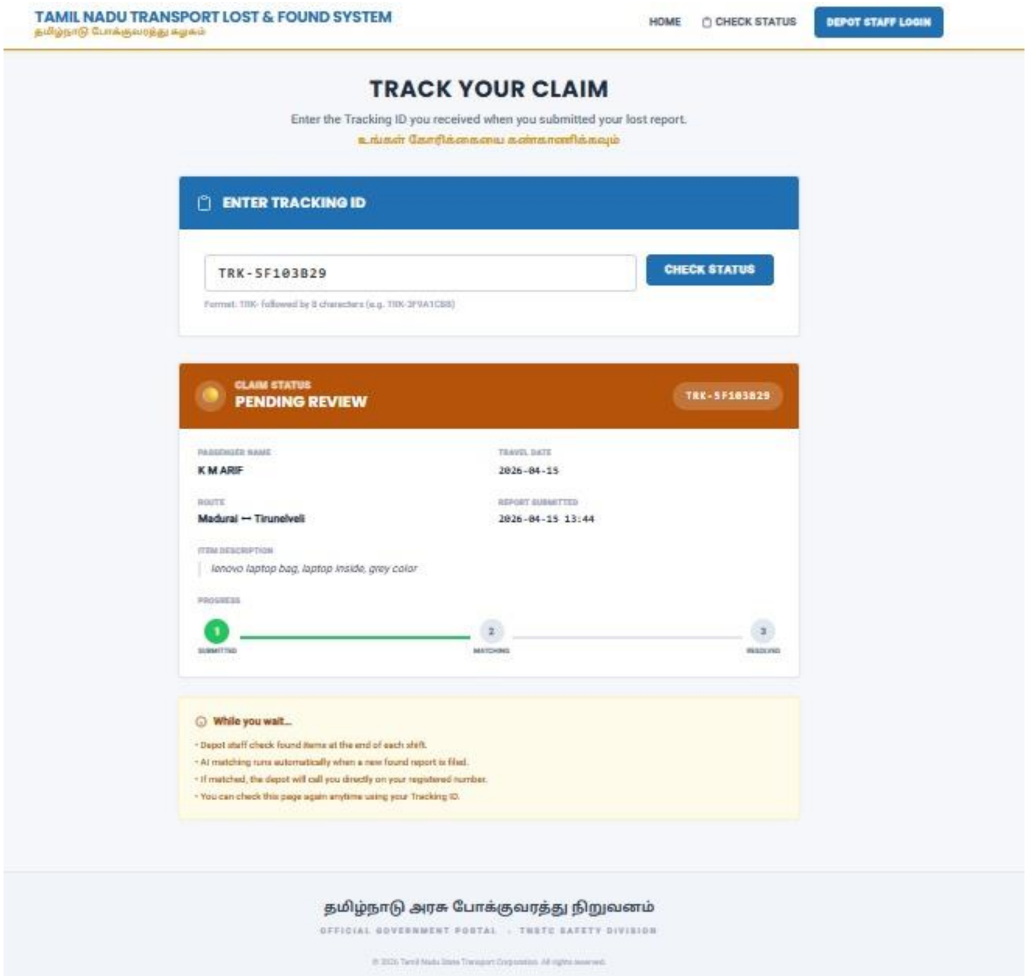

Fig 3.3: Pending Status of the product with Tracking ID

3.1.2 Depot Staff Authentication

Depot staff access the system through a secure login portal. Staff members select their depot from a dropdown and authenticate using a password. The login page, shown in Figure 3.3, is restricted to authorized personnel only, as indicated by the 'Authorized Personnel Only' notice at the bottom. Session-based authentication is managed by Flask to ensure only verified depot staff can access the dashboard.

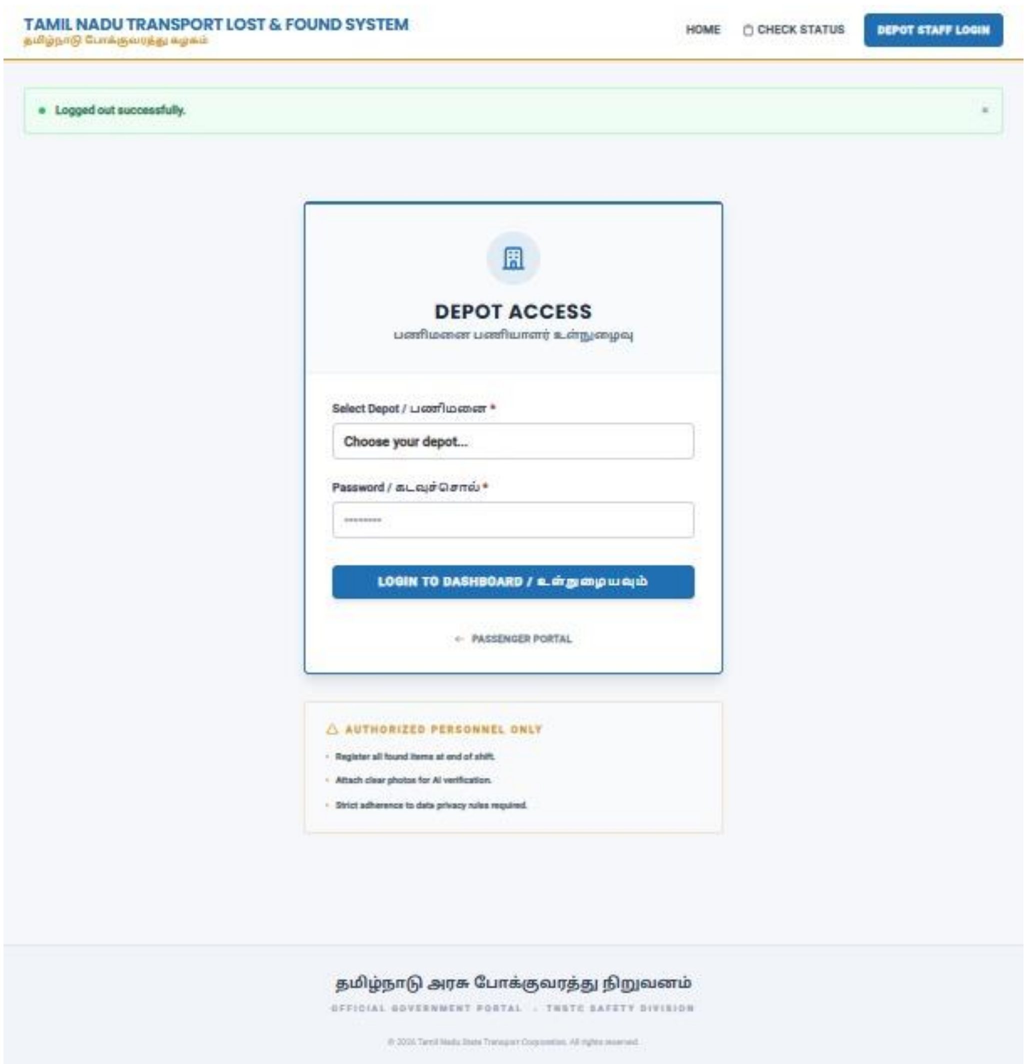

Fig 3.3: Depot Staff Login Page

3.1.3 Depot Staff Found-Item Reports

Once authenticated, depot staff are presented with the depot dashboard. Before any found items are logged, the right panel displays a 'No Matches Yet' placeholder with a prompt to register found items, as seen in Figure 3.4. The dashboard also shows an interactive depot route map, recent activity logs, and the 'Register Found Item' form on the left, where staff enter the route, date found, a description of the item, and upload a photograph.

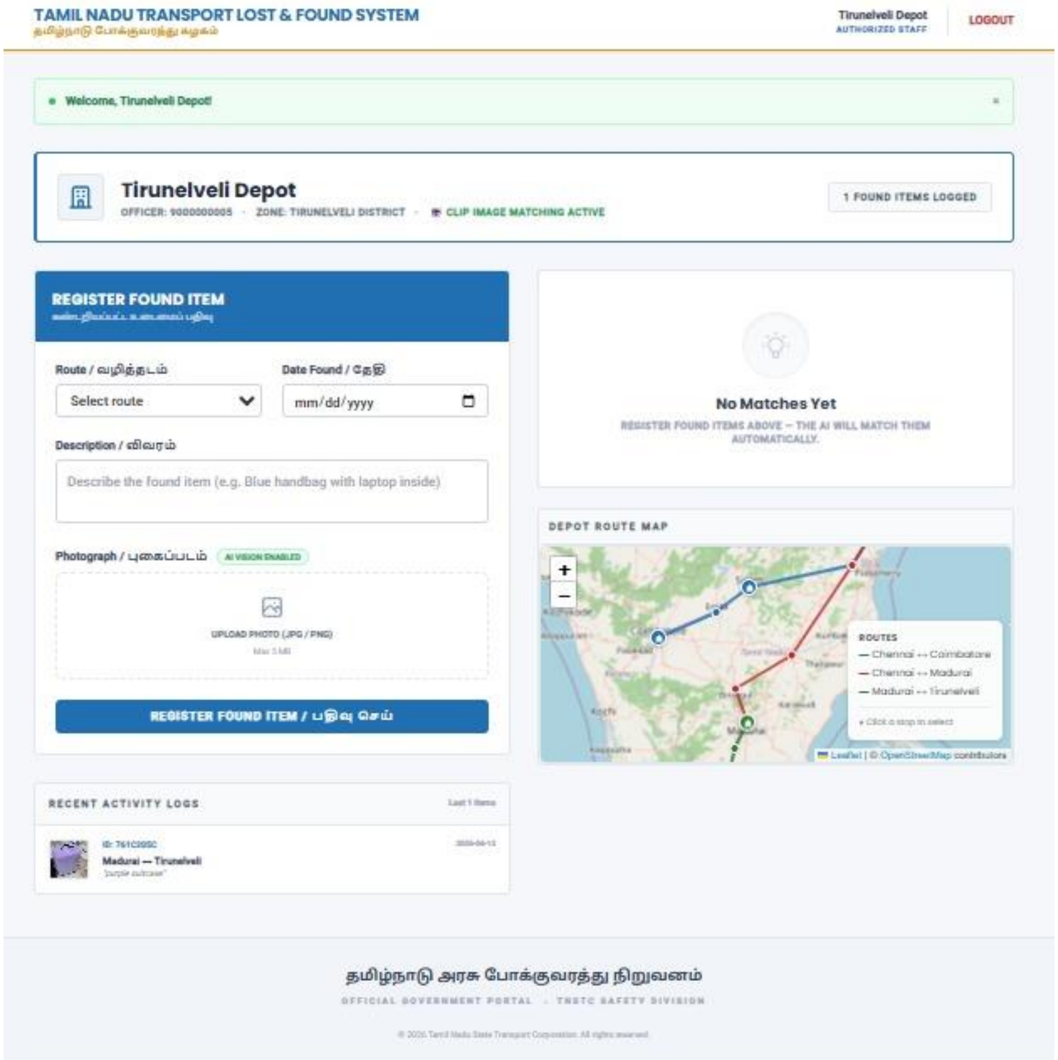

Fig 3.4: Depot Dashboard — Awaiting Found Item Registration

3.2 Data Preprocessing

Data preprocessing prepares the raw text inputs for the AI engine. Text inputs — passenger descriptions and depot notes — are cleaned and normalized before being passed to the Sentence-BERT encoder. A memory-cache mechanism is implemented in similarity.py so that repeated encoding of identical text strings is avoided, significantly reducing computation time in high-traffic scenarios. The route data stored in MongoDB is represented as ordered arrays of stop names, which serve as the basis for the luggage path filter.

3.3 Model Architecture and Design

The system is built on a classic client-server architecture backed by a NoSQL database. The following describes each layer and component.

3.3.1 Frontend (Client Layer)

The user interface is built with HTML, CSS, and JavaScript. The passenger-facing side includes an interactive map powered by Leaflet.js that displays the selected route visually, making the form intuitive to complete. The staff-facing dashboard is accessible only after login, showing found items and AI match results.

3.3.2 Backend (Server Layer)

A Flask (Python) server acts as the central controller. It defines all web routes (e.g., /lost, /depot), handles form submissions and file uploads, manages session-based authentication for depot staff, generates UUIDs for tracking IDs, and coordinates all database reads and writes. The backend is intentionally stateless — no critical data is stored in server memory between requests.

3.3.3 AI Engine (similarity.py)

The AI engine is a dedicated Python module that handles all similarity computations. It uses Sentence-BERT to encode text descriptions into embeddings and computes cosine similarity between the passenger and depot text embeddings. The Unified Scorer combines this text similarity with route validity to produce a final confidence score. Any pair scoring above a threshold of 0.30 is saved as a potential match in the MongoDB matches collection.

3.3.4 Database (MongoDB)

A MongoDB NoSQL database stores all lost reports, found item records, match results, route definitions, and depot credentials. MongoDB is selected because lost and found item documents have flexible, schema-less structures — the image path field, for example, is optional in lost reports but mandatory in found reports. JSON-like documents handle this variability naturally without requiring schema migrations.

3.3.5 One-Time Match Computation

When a depot staff member submits a new found-item report, the backend immediately triggers a match computation pass (compute_and_save_matches). This function iterates over all pending lost-item reports, applies the luggage path filter, computes the unified score for each valid candidate, and saves qualifying matches to the database. This design ensures that the match dashboard loads instantly for staff, as all computations are performed at submission time rather than at display time.

CHAPTER 4

RESULTS AND ANALYSIS

4.1 System Performance Comparisons

The system was evaluated across a set of simulated lost-and-found scenarios using real TNSTC route data seeded into the database. Test cases were constructed with varying degrees of description similarity — ranging from near-identical descriptions to scenarios where only synonymous vocabulary was used (e.g., 'backpack' vs 'rucksack'). The Sentence-BERT scorer consistently returned higher match scores for semantically similar descriptions than for unrelated ones, demonstrating the effectiveness of the embedding approach over rigid keyword matching.

In terms of geographic filtering, the luggage path logic successfully eliminated all depot candidates positioned before the passenger's alighting stop on the route array, producing zero false positives from geographically impossible locations.

4.2 Effectiveness of the Unified Scoring Approach

The two-factor weighted scoring model combines text semantic similarity with route validity. Route validity acts as a binary gating factor an item at an impossible depot location receives a route score of zero, which suppresses its overall match score below the 0.30 threshold even when text similarity is moderate. Text similarity carries the primary weight as it is the most consistently provided data point. The match confidence percentage is clearly displayed on the depot dashboard to assist staff in prioritizing review.

4.3 Impact of AI Matching on Recovery Efficiency

Without AI-assisted matching, depot staff must manually compare every found item against every pending lost report — an O(n^2) manual task that becomes infeasible as the number of reports grows. The automated match computation, triggered at found-item submission time, reduces the staff's role to a simple physical verification step: they review the AI-ranked match cards sorted by confidence percentage and click 'Resolve' once they confirm a match. This dramatically reduces the cognitive and time burden on depot staff and accelerates the resolution time from days to minutes.

4.4 Visualization of Results

When a depot staff member registers a found item and a matching lost report exists, the system immediately computes the match and displays it under the 'Potential Matches Found' panel on the dashboard. Figure 4.1 shows the depot dashboard after a match has been identified. The panel displays the found item thumbnail, route, date, the matched passenger's tracking ID and name, contact number, and the computed score breakdown (Text, Route, and Final percentage).

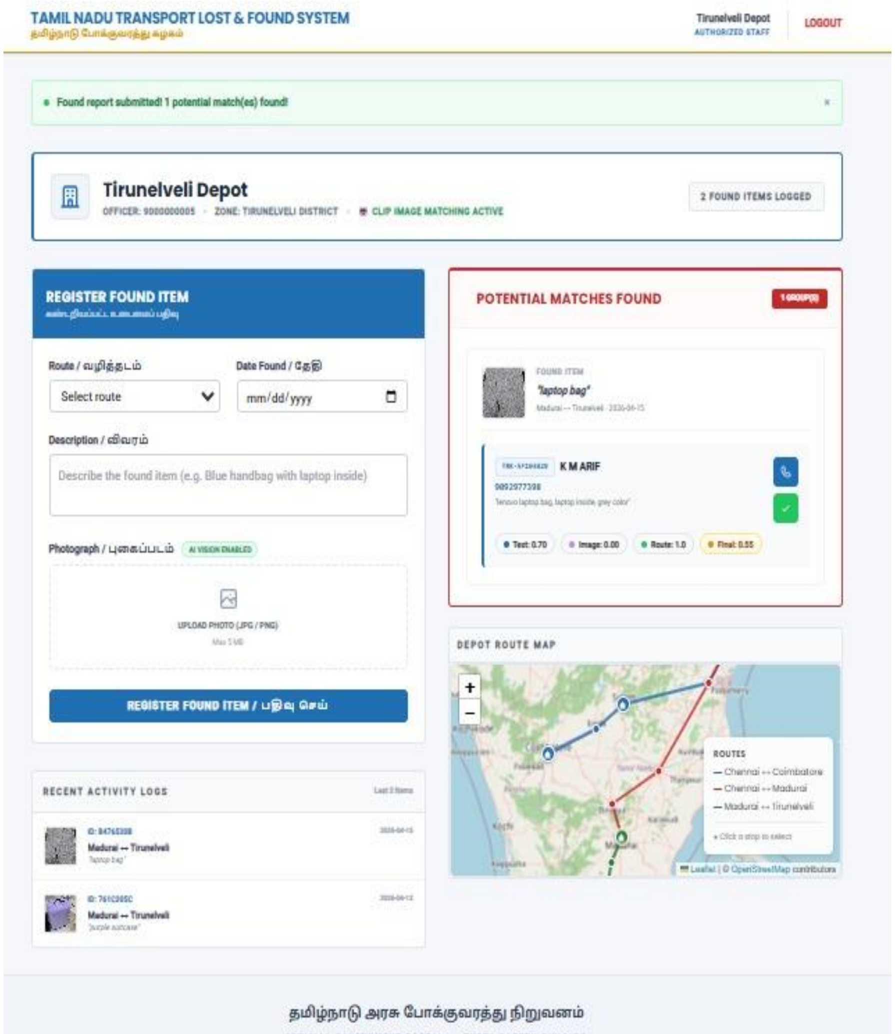

Fig 4.1: Depot Dashboard Showing Potential Match Found with AI Score Breakdown

Passengers can track the progress of their claim at any time using their Tracking ID. The status page shows a three-stage progress indicator: Submitting, Matching, and Resolved.

Once the depot staff physically verifies the item and clicks 'Resolve', the passenger's tracking page updates to display the 'Resolved' status along with the exact matched depot name. Figure 4.2 shows the fully resolved claim, where the passenger is informed to collect their item from Coimbatore Depot, completing the end-to-end recovery workflow.

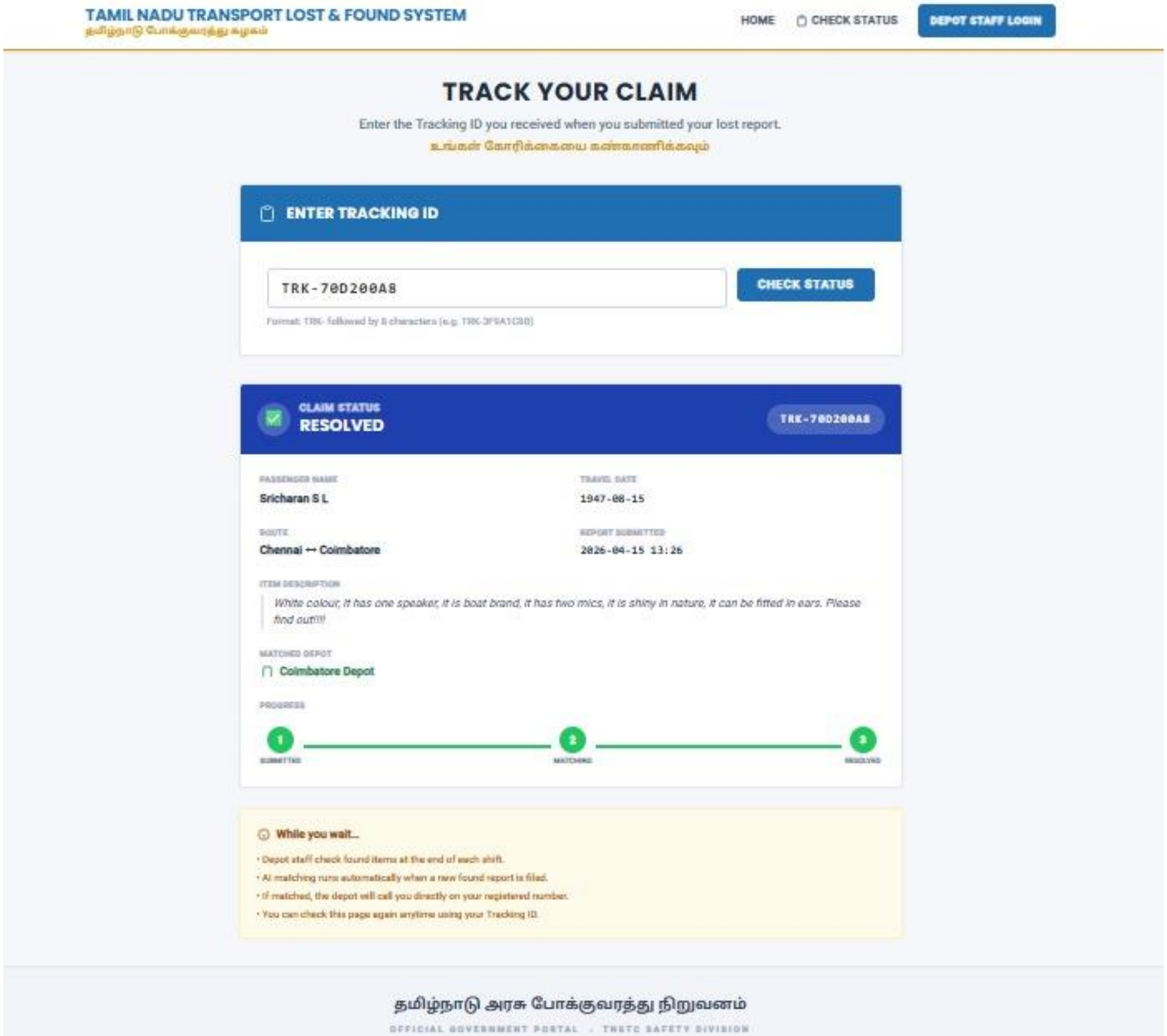

Fig 4.2: Passenger Tracking Page — Claim Status: Resolved (Coimbatore Depot)

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Interpretation of Results

This project successfully demonstrates that AI-driven semantic matching can be applied to solve a real-world civic problem in public transport management. The combination of Sentence-BERT for text understanding and a custom directional luggage path filter produces a system that is meaningfully more accurate, faster, and scalable than any manual or keyword-based approach. The unified scoring model provides a robust, interpretable confidence metric that helps staff quickly identify true matches while filtering out false candidates. The one-time match computation architecture ensures that the system remains performant even as the volume of reports grows, because the expensive AI computation is amortized at submission time rather than at query time.

The platform is deployable by any state transport organization with minimal infrastructure requirements — a modest server running Python and MongoDB is sufficient. The modular design of the codebase (app.py, similarity.py, seed_db.py) makes it straightforward to extend or adapt the system to other transport modes such as railways or urban metro networks.

5.2 Limitations and Considerations

The current system has several limitations that should be acknowledged. First, the 0.30 match threshold was determined empirically and may require tuning based on real deployment data. A threshold that is too low will generate too many false positives for staff to review, while a threshold that is too high risks missing genuine matches.

Second, the system currently supports only pre-seeded routes. Dynamic route management — adding or modifying routes without rerunning seed_db.py — is not yet supported. Future work could address this through an admin interface for route management. Third, the system does not yet provide automated passenger notifications via SMS or email when a match is found. Integrating a notification service such as Twilio (SMS) or an SMTP email provider would significantly improve the passenger experience. Finally, multilingual support for regional languages such as Tamil would further enhance accessibility for the intended user base.

REFERENCES

1. Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics, pp. 3982-3992.

2. Wolf, T., Debut, L., Sanh, V., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 38-45.

3. MongoDB, Inc. (2023). MongoDB Documentation: Introduction to MongoDB. Available at: https://www.mongodb.com/docs/. Accessed April 2025.

4. Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python, 2nd edition. O'Reilly Media, Sebastopol, CA.

5. Aggarwal, C. C. and Zhai, C. X. (2012). A Survey of Text Classification Algorithms. Mining Text Data. Springer, Boston, MA, pp. 163-222.

6. Leaflet.js Contributors (2024). Leaflet: an open-source JavaScript library for mobile-friendly interactive maps. Available at: https://leafletjs.com. Accessed April 2025.